

AgentStudio — Product Requirements Document

Product Name: AgentStudio

Version: 1.0

Status: Draft

Author: Kartik Manimuthu

Date: May 2026

Executive Summary

AgentStudio is a low-code, visual platform that enables developers and non-developers alike to design, configure, test, and deploy production-grade AI agents built on top of LangGraph. It combines the visual intuitiveness of Langflow with a more opinionated, LangGraph-native architecture — enabling stateful, multi-agent, graph-based workflows with deep RAG/knowledge base support, a live playground, and one-click deployment. The primary target audience is AI engineers and product teams at companies that want to ship agents fast without losing control of the underlying infrastructure.

Problem Statement

Building LangGraph agents today requires writing Python boilerplate to define nodes, edges, state schemas, and checkpoints. Hooking in RAG pipelines, tool configurations, memory, and multi-agent routing compounds the complexity. Teams waste days setting up infrastructure instead of iterating on agent logic. Existing visual tools like Langflow are LangChain-first and don't expose LangGraph's stateful graph primitives natively. There is no product today that gives teams a visual-first, LangGraph-native agent builder with integrated knowledge base management, live testing, version control, observability, and one-click deployment.

Goals & Success Metrics

Goals

- Reduce time-to-first-agent from days to under an hour for an AI engineer
- Make multi-agent, stateful graph configuration possible without writing Python
- Provide a production-grade platform with observability, versioning, and API deployment built in

Success Metrics (12 months)

- ≥ 500 active organizations within 12 months of launch
- Median time to deploy first agent ≤ 45 minutes
- ≥ 70% of deployed agents using at least one knowledge base
- < 2% agent deployment failure rate
- ≥ 85% playground-to-deployment conversion rate

Target Users

Persona	Description	Primary Use Case
AI Engineer	Builds and fine-tunes agents professionally	Visual graph builder, code escape hatches, LangGraph export
Full-Stack Developer	Integrates agents into existing products	API deployment, webhook triggers, SDK access
Product Manager / Domain Expert	Configures agents without code	No-code agent templates, playground testing
Enterprise AI Team	Manages multiple agents across business units	Multi-workspace, RBAC, audit logs, SSO

Feature Specification

1. Visual LangGraph Canvas

The core of AgentStudio is a node-and-edge visual editor that maps directly to LangGraph's graph primitives.

Nodes (first-class visual components):

- **LLM Node** — configure model provider (OpenAI, Anthropic, Bedrock, Ollama, etc.), model ID, temperature, max tokens, stop sequences, system prompt
- **Tool Node** — attach one or more tools (built-in or custom); shows tool call/response in the graph
- **Router / Conditional Edge Node** — define routing logic via a natural-language condition or Python function; renders as a diamond in the canvas
- **Retriever Node** — connect to a knowledge base (vector store, document store); configure top-k, similarity threshold, reranking
- **Memory Node** — short-term (thread-scoped buffer), long-term (persistent user-level store), summarization memory types
- **State Schema Node** — define the TypedDict-shaped state that flows through the graph; field-level type annotations and default values

- **Human-in-the-Loop Node** — pause execution for human approval/input; configurable timeout and fallback behavior
- **Subgraph Node** — embed another AgentStudio graph as a reusable subagent
- **Custom Python Node** — write arbitrary Python code inline; full SDK access to state, tools, and services
- **API Call Node** — configure REST/GraphQL calls with auth, headers, retry logic, and response mapping
- **Webhook Node** — expose graph entry/exit points as webhooks with secret-based auth

Canvas Interactions:

- Drag-and-drop from a categorized component sidebar
- Connect nodes by drawing edges; label edges with conditions
- Zoom, pan, minimap, auto-layout
- Group nodes into visual clusters / subgraph regions
- Inline commenting on nodes and edges
- Undo/redo with full history

Code Export / Import:

- Export the entire graph as runnable LangGraph Python code (TypedDict state, compiled graph, tool definitions)
- Import an existing `.py` LangGraph file and have it parsed into the canvas
- Two-way sync: edits in canvas reflect in code and vice versa (Git-tracked)

2. Agent Configuration Panel

Each agent has a structured configuration that is separate from the graph topology.

General Settings:

- Agent name, description, tags, version
- Agent type: ReAct, Plan-and-Execute, Multi-Agent Supervisor, Custom
- Entry node and termination conditions (max iterations, token budget, END node)
- Recursion limit for graph traversal

LLM Provider Configuration:

- Support for: OpenAI, Anthropic, AWS Bedrock, Azure OpenAI, Google Vertex AI, Groq, Together AI, Ollama (local), LiteLLM proxy, custom OpenAI-compatible endpoint
- Per-node model override vs. global model setting
- Model fallback chain (primary → fallback → last resort)
- Cost cap per run (abort if estimated cost exceeds threshold)

Prompt Management:

- System prompt editor with variable interpolation ({{user_name}}, {{context}})
- Few-shot example manager per node
- Prompt versioning and A/B variant support
- Jinja2 templating for dynamic prompts

Tool Registry:

- Built-in tools: web search (Tavily, Brave, Serper), code interpreter (sandboxed Python), calculator, file reader (PDF, CSV, DOCX), SQL query runner, HTTP request, calendar access
- Custom tool builder: define function signature, parameter schema (JSON Schema), implementation (Python or API call)
- MCP (Model Context Protocol) client support — connect to any MCP server as a tool source
- Tool-level authorization: which nodes can call which tools
- Tool call retry policy and timeout configuration

Memory & State Configuration:

- Thread-scoped checkpointing provider: PostgreSQL, DynamoDB, Redis, SQLite (local dev)
- Long-term memory: vector-based user memory with semantic search
- Session context window: configure how many past messages to include
- Memory summarization trigger: token threshold or turn count

Multi-Agent Configuration:

- Supervisor agent + worker agent topology
- Define handoff conditions: which agent hands off to which, under what state conditions
- Agent communication protocol: shared state vs. message passing
- Parallel agent execution for non-dependent branches
- Agent isolation: each worker agent can have its own model, tools, and memory config

3. Knowledge Base Management

A dedicated module for creating, managing, and connecting knowledge bases to agent retriever nodes.

Data Ingestion:

- Upload sources: PDF, DOCX, TXT, CSV, JSON, XLSX, Markdown, HTML
- URL scraping: single page, sitemap-based crawl, recursive crawl with depth control

- Connector integrations: Notion, Confluence, Google Drive, SharePoint, Slack (export), GitHub (repos, issues, PRs), Jira
- Scheduled re-sync for all connectors (configurable interval: hourly to weekly)
- Incremental sync — only re-embed changed/new documents

Chunking & Processing Pipeline:

- Chunking strategies: fixed-size, recursive character, semantic (sentence-level), markdown-aware, code-aware
- Chunk size and overlap configuration
- Pre-processing steps: HTML stripping, PII redaction (configurable), OCR for scanned PDFs, table extraction
- Custom pre-processing Python hook

Embedding Configuration:

- Embedding model: OpenAI text-embedding-3-small/large, Cohere, AWS Bedrock Titan, local models (BGE, E5, Nomic via Ollama), custom endpoint
- Embedding dimensions and normalization settings

Vector Store Backends:

- Managed (hosted by AgentStudio): pgvector on Aurora, OpenSearch
- BYOC (Bring Your Own): Pinecone, Weaviate, Qdrant, Chroma, FAISS, Milvus
- Connection config stored as encrypted secrets

Retrieval Configuration (per Retriever Node):

- Top-k results
- Similarity threshold filter
- Metadata filters (by source, date, tag, document type)
- Reranking: Cohere Rerank, cross-encoder, none
- Hybrid search: dense + sparse (BM25) with configurable alpha weight
- Contextual compression: LLM-based post-retrieval filter

Knowledge Base Testing:

- Built-in retrieval tester: type a query, inspect returned chunks with scores
- Chunk browser: browse all indexed content with full metadata
- Embedding visualization (2D UMAP projection)

4. Playground

A live, interactive environment to test agents end-to-end before deployment.

Chat Interface:

- Multi-turn conversation UI with markdown rendering
- Streaming responses (token-by-token display)
- Attach files for per-session RAG testing
- Switch active agent version mid-conversation
- System prompt override for rapid iteration

Graph Execution Trace:

- Side-by-side trace panel: see which nodes fired in what order
- Per-node view: input state → tool calls → output state
- LangGraph checkpoint state displayed at every step (raw JSON, formatted view)
- Highlight the active edge/node in the canvas as the trace plays back

Debug Tools:

- Step-by-step execution: pause after each node, inspect state, manually override before continuing
- Inject arbitrary state at any node for targeted testing
- Replay a past conversation turn with different state/config
- Error root-cause: exception stack traces mapped back to the specific node

Session Management:

- Named test sessions with save/restore
- Export session transcript as JSON or Markdown
- Share session link with team members (read-only or interactive)

Variant Testing:

- Run two agent configurations in parallel (A/B mode) on the same input
- Side-by-side output comparison with latency, token count, and cost comparison

5. Deployment & API Gateway

One-click deployment pipeline from canvas to production endpoint.

Deployment Targets:

- Managed cloud (AgentStudio-hosted): containerized on ECS/Fargate; auto-scaled
- AWS deployment to user's own account via CDK/Terraform export

- Docker export: generate Dockerfile + `docker-compose.yml` for self-hosting
- LangGraph Cloud (official LangChain Cloud) push with one click

Endpoint Types:

- REST API: `POST /invoke` (sync), `POST /stream` (SSE streaming), `POST /batch`
- WebSocket endpoint for stateful sessions
- Webhook trigger: fire graph on incoming webhook with configurable mapping
- Scheduled trigger: cron-based graph invocation

API Management:

- API key creation and rotation per agent
- Rate limiting: per-key, per-agent, per-org
- Request/response logging with configurable retention
- IP allowlist and CORS configuration
- SDK generation: Python, TypeScript, Go (auto-generated from agent schema)

Versioning & Rollout:

- Semantic versioning per agent (major.minor.patch)
- Blue/green deployment with traffic split control (10%, 50%, 100%)
- Instant rollback to any previous version
- Promote from playground → staging → production environments

6. Observability & Monitoring

Deep runtime visibility without external instrumentation needed.

Tracing:

- Every agent invocation captured as a full LangSmith-compatible trace
- Native Langfuse integration for open-source observability
- OpenTelemetry export to Datadog, Jaeger, Honeycomb, Grafana Tempo

Metrics Dashboard:

- Invocation volume, success/error rates, P50/P95/P99 latency
- Token consumption per model, per agent, per user
- Tool call frequency and failure rates
- Retrieval hit rate, mean chunk relevance score
- Cost breakdown: per-run, per-day, per-agent, per-team

Alerting:

- Threshold alerts: latency, error rate, cost spike

- Anomaly detection: sudden traffic or error spikes
- Delivery channels: email, Slack, PagerDuty, webhook

Evaluation:

- Built-in eval harness: upload test dataset (input/expected output pairs), run against any agent version
- LLM-as-judge scoring: correctness, faithfulness (for RAG), relevance, toxicity
- Regression tracking: compare eval scores across versions
- Human feedback collection: thumbs up/down in playground, annotation UI

7. Flow Control & Orchestration

Beyond basic agent graphs, AgentStudio supports complex orchestration patterns.

Flow Components:

- Map Node — apply a subgraph to every item in a list, in parallel or sequential
- Reduce Node — aggregate results from parallel branches
- Loop Node — iterate over a graph until a termination condition is met (with safety limits)
- Wait/Join Node — synchronize parallel branches before continuing
- Delay Node — insert configurable wait time (useful for rate-limited APIs)
- Switch Node — multi-branch conditional routing (like a switch-case)

External Event Integration:

- EventBridge / Kafka / SQS triggers
- Long-running agent with async human-in-the-loop via email or Slack
- Resume a paused graph via API call or button click in a UI

8. Templates & Component Marketplace

Built-in Templates:

- Customer support agent with RAG + escalation
- Research agent (web search + summarization + citation)
- SQL data analyst agent
- Code review agent (GitHub integration)
- Document Q&A agent (knowledge base + citation)
- Multi-agent pipeline: researcher → writer → reviewer
- Email triage and drafting agent
- Voice agent (Whisper STT + TTS output)

Component Marketplace:

- Community-contributed custom nodes (reviewed and signed)
- Publish your own custom node/tool to the marketplace
- Install any community component with one click
- Versioning and deprecation support for marketplace components

9. Collaboration & Workspace

Team Workspaces:

- Organization → Workspace → Project → Agent hierarchy
- Role-based access control: Owner, Admin, Developer, Viewer
- Per-project API key and secrets management
- Shared component library within a workspace

Version Control:

- Git-backed agent definitions (JSON + Python export)
- Native GitHub/GitLab integration: PR-based review flow for agent changes
- Branch-per-feature workflow with merge and diff views
- Change audit log: who changed what and when

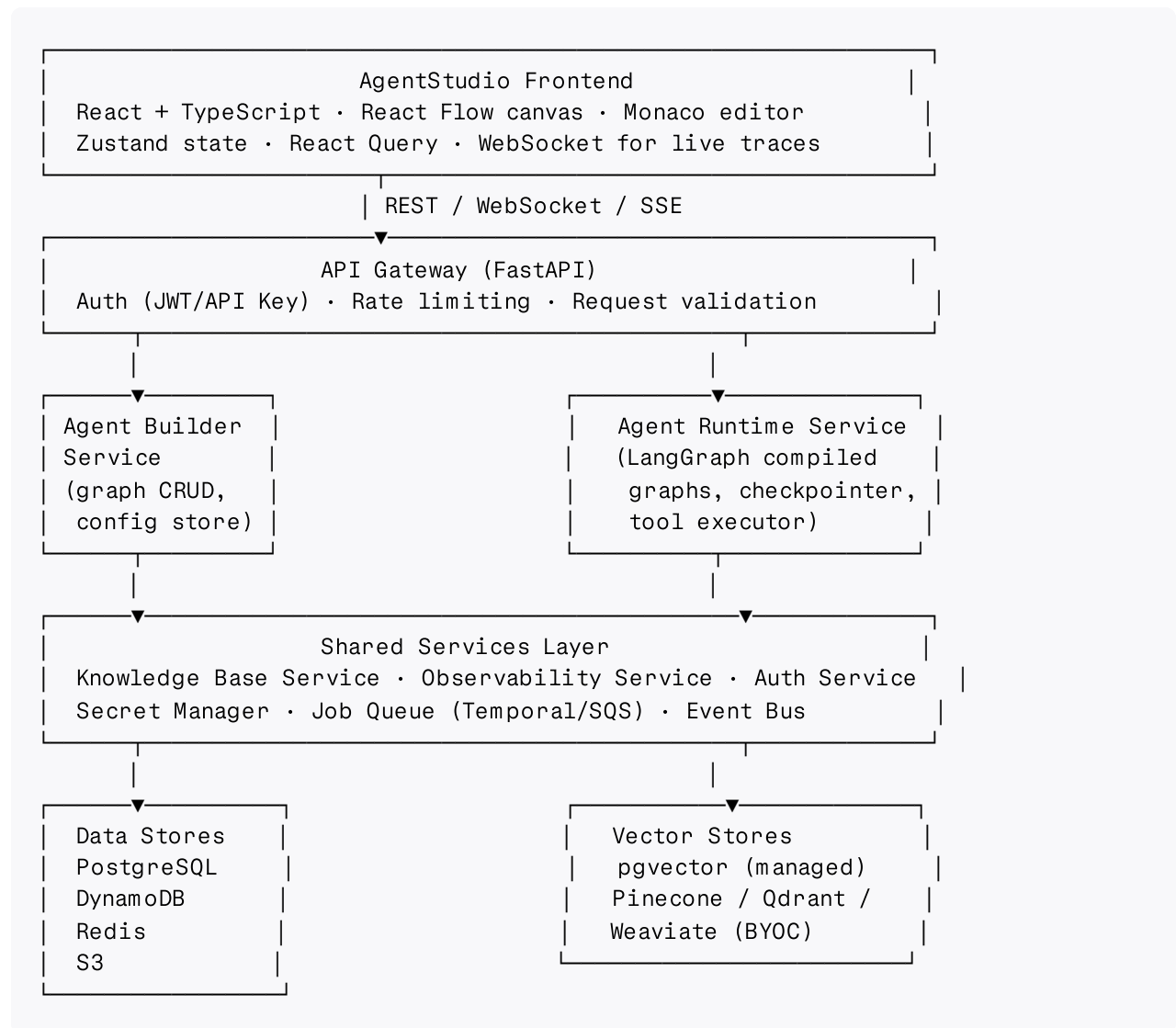
Comments & Reviews:

- Inline commenting on nodes, edges, and configuration fields
- Review requests before promoting to production
- Notification center (in-app, email, Slack)

10. Security & Compliance

- Secrets management: all API keys, DB passwords, and tokens stored encrypted (AES-256) using AWS Secrets Manager or HashiCorp Vault
- Secret injection at runtime — never exposed in logs or traces
- SOC 2 Type II compliance target (Year 1)
- SSO via SAML 2.0 / OIDC (Okta, Azure AD, Google Workspace)
- Audit logs: all configuration changes, deployments, and API access
- VPC-isolated deployment option for enterprise (bring your own VPC)
- PII redaction in traces (configurable via regex or LLM-based detection)
- Data residency: EU, US, AP region selection
- Agent sandboxing: code interpreter node runs in ephemeral, network-isolated container

Technical Architecture Overview



Frontend: React + TypeScript, React Flow (canvas), Monaco Editor (code nodes), Zustand (state), TailwindCSS

Backend: Python FastAPI, LangGraph (agent runtime), Celery + Redis (background jobs), Temporal (durable workflows for long-running agents)

Infrastructure: AWS ECS/Fargate, RDS Aurora PostgreSQL, ElastiCache Redis, S3, CloudFront, WAF, AWS CDK

Observability: OpenTelemetry → Grafana/Langfuse; structured logging to CloudWatch / OpenSearch

Out of Scope for v1.0

- Fine-tuning UI (model training, LoRA adapters) — v2 roadmap
- Native mobile app — web responsive only
- Built-in billing for end-users of deployed agents (metered API) — v2
- Real-time collaborative canvas editing (multi-cursor) — v2
- GraphQL API — REST + WebSocket sufficient for v1

Risks & Mitigations

Risk	Likelihood	Impact	Mitigation
LangGraph API breaking changes	Medium	High	Pin LangGraph version; automated upgrade tests
Security vulnerability in Code Node	Medium	Critical	Sandboxed container execution; network isolation; no filesystem access
Vendor LLM API outages	High	Medium	Model fallback chain; multi-provider routing
Canvas complexity overwhelming non-technical users	Medium	Medium	Opinionated templates; guided wizard for first agent; progressive disclosure
Data privacy in traces (PII leakage)	Medium	High	PII redaction at trace ingestion; field-level masking config

Roadmap

Phase 1 — MVP (Months 1–4)

- Visual canvas: LLM, Tool, Router, Retriever, Custom Python nodes
- Agent configuration panel (LLM providers, prompts, tools)
- Knowledge base (file upload + URL, pgvector, OpenAI embeddings)
- Playground (chat, basic trace view)
- REST API deployment (managed)
- User auth, workspaces, API keys

Phase 2 — Production Ready (Months 5–8)

- Memory node (short + long-term)
- Multi-agent supervisor topology
- Human-in-the-loop node
- Full observability (Langfuse, metrics dashboard, alerting)
- Evaluation harness
- GitHub/GitLab integration
- Docker and AWS CDK export

Phase 3 — Scale & Ecosystem (Months 9–12)

- Component marketplace (community nodes)
- MCP server support (expose agent as MCP)
- Advanced flow control (Map/Reduce, Loop)

- Enterprise features: SSO, RBAC, VPC isolation, audit logs
- Connector integrations: Notion, Confluence, Google Drive, Slack
- A/B variant testing in playground

Phase 4 — v2 Directions

- Fine-tuning UI (LoRA/QLoRA on open-source models)
- Real-time collaborative canvas
- Voice agent builder (STT/TTS integration)
- Multimodal nodes (vision, document understanding)
- Metered API billing for end-users of deployed agents

Competitive Differentiation

Feature	AgentStudio	Langflow	LangSmith	Flowise
LangGraph-native visual canvas	✔ First-class	⚠ Partial (LangChain-first)	✖	✖
Graph state schema editor	✔	✖	✖	✖
Stateful checkpointing config	✔	✖	✖	✖
Built-in knowledge base management	✔	⚠ Basic	✖	⚠
Live graph execution trace in playground	✔	⚠	✔	✖
A/B variant testing	✔	✖	✔	✖
Human-in-the-loop node	✔	✖	✖	✖
Multi-agent supervisor topology	✔	⚠	✖	✖
Code export (runnable Python)	✔	✔	✖	✔
MCP client + server	✔	✔	✖	✖
Built-in eval harness	✔	✖	✔	✖
Self-hostable	✔	✔	✖	✔

Appendix: Suggested Additional Features (Not Yet Scoped)

The following features were identified as valuable additions beyond the initial concept:

1. **Prompt Versioning & A/B Testing** — Track every prompt change, run statistical significance tests on output quality
2. **Agent Cost Estimator** — Before deployment, estimate monthly cost based on usage patterns

3. **Guardrails Layer** — Built-in input/output validation: PII filters, toxicity detection, topic restriction, length limits
4. **Simulation Mode** — Mock tool calls with user-defined fixtures to test agent logic without real API calls
5. **Token Budget Manager** — Visual indicator and hard stop when agent exceeds per-run token budget
6. **Multi-Language SDK** — Auto-generated clients in Python, TypeScript, Go, Java from the agent's OpenAPI schema
7. **Changelog & Diff View** — Visual diff between two agent versions: which nodes changed, which prompts changed
8. **End-User Feedback Collection** — Embed a thumbs-up/down widget in your app; feedback flows back to AgentStudio evals
9. **SLA Monitoring** — Define uptime and latency SLAs per agent; automated incident creation on breach
10. **Trace Replay** — Pick any historical production trace and replay it in the playground with the same inputs
11. **Voice Agent Builder** — Connect Whisper (STT) and a TTS provider; build voice-first agent workflows visually
12. **Agent Marketplace** — Publish complete, configured agents as templates for others to clone and adapt
13. **Webhook Debugger** — Inspect, replay, and simulate incoming webhooks that trigger agents (like Ngrok/Webhook.site built-in)
14. **RBAC for Tools** — Restrict which user roles or agent types can invoke sensitive tools (SQL, file access, external APIs)
15. **Cost Allocation Tags** — Tag agents, runs, and knowledge bases by team/project for internal cost chargeback